Technology
Arts Sciences
TH Köln

PenFlow

PAPER

prepared by

Collin Seibt | 1146471
Andreas Schurawlev | 11138619

Academic work as part of the
Master's Program (M.Sc.)

Submitted to

Technical University of Cologne
Campus Gummersbach
Faculty of Computer Science and
Engineering Sciences

in the degree program
Media Informatics

in the module: Ubiquitous Computing

Cologne, Winter Semester 2024/25

Abstract

Handwriting digitization in real time has been a subject of extensive research, particularly with the rise of interactive applications in education, communication, and accessibility. Initial explorations using inertial measurement units (IMUs) showed promise in capturing pen motion but were ultimately hindered by drift and error accumulation. This paper documents our transition from IMU-based tracking to optical sensor-based tracking, analyzing the shortcomings of IMUs, the advantages of optical solutions, and presenting a viable approach for real-time handwriting digitization.

Contents

1 Introduction **4**

2 Related Work **5**
2.1 Towards an IMU-based Pen Online Handwriting Recognizer 5
2.2 Deep-Learning-Based Character Recognition from Handwriting Motion Data Captured Using IMU and Force Sensors 6

3 IMU-Based Handwriting Tracking **7**
3.1 Sensor Selection and Setup 7
3.1.1 IMU-to-ESP32 Connection 7
3.1.2 Data Acquisition Code 8
3.1.3 Example Raw Data Output 9
3.2 Data Processing and Tracking Algorithm 9
3.2.1 IMU Testing with Pygame Visualization 10
3.2.2 Implementation of Data Processing 11
3.2.3 Challenges and Limitations 20

4 Transition to optical sensor tracking **21**

5 Conclusion and Future Work **23**

1 Introduction

Handwriting is one of the most fundamental forms of human communication, and its digitization has become an essential component in modern technology. Digital handwriting recognition is widely used in applications such as note-taking, signature verification, and assistive technology for individuals with disabilities. Traditionally, handwriting recognition relies on optical tracking, touchscreen input, or pressure-sensitive surfaces. However, these methods require specialized hardware, making them less accessible for general use.

IMU-based handwriting tracking offers an alternative approach that relies solely on motion sensors to capture pen movements. By using accelerometers, gyroscopes, and magnetometers, an IMU can estimate movement trajectories without requiring an external tracking surface. This technique has the potential to enable seamless handwriting digitization on any surface. However, reconstructing position from IMU data is challenging due to error accumulation and sensor noise.

This study seeks to answer the following research question:

„How can we accurately transfer handwriting created with a physical pen into a digital format in real time?“.

To address this question, we implemented and tested different sensor fusion techniques for IMU-based motion tracking. Our approach involved integrating an MPU9250 IMU with an ESP32 microcontroller, developing a data processing pipeline using an Extended Kalman Filter (EKF), and visualizing the reconstructed handwriting trajectory. We explored real-time processing using Python and post-processing using MATLAB, assessing the accuracy and feasibility of IMU-based handwriting tracking.

Our experiments revealed substantial limitations in IMU-based tracking due to extreme drift and cumulative errors, ultimately leading us to explore an alternative optical tracking method.

2 Related Work

Various approaches exist for combining digital and analogue writing. Most use special surfaces to capture pen movements, as with graphic tablets. However, we found two research projects that digitally capture handwriting without a special surface, using technology built into the pen.

2.1 Towards an IMU-based Pen Online Handwriting Recognizer

The Paper „Towards an IMU-based Pen Online Handwriting Recognizer“ presents a handwriting recognition system using IMU-sensors (inertial measurement units) to digitize the text written on a paper. In addition the system uses a LSTM neural network to interpret the sensor data (cf. Wehbi et al. (2021)). Figure 2.1 shows the

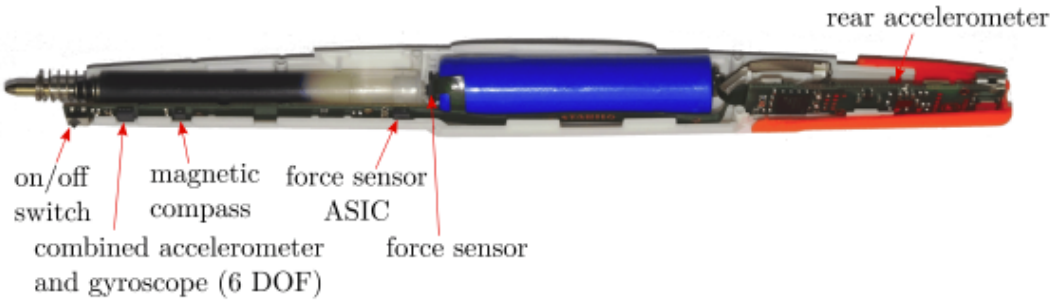


Figure 2.1: Sensor Placement in the Pen from Wehbi et al. (2021)

pen, which is equipped with a force sensor, a magnetic compass, several accelerometers and a gyroscope, which are needed to digitize the data written on a piece of paper.

Strengths and Weaknesses

The strength of this solution lies in the combination of sensors, which reduces the dependency on external surfaces or specialized tools. On the other hand these sensors need precise calibration and increase hardware complexity which is a major weakness.

2.2 Deep-Learning-Based Character Recognition from Handwriting Motion Data Captured Using IMU and Force Sensors

The Paper „Deep-Learning-Based Character Recognition from Handwriting Motion Data Captured Using IMU and Force Sensors“ presents a digital pen that is able to recognize 36 alphanumeric characters utilizing deep-learning (cf. Alemayoh et al. (2022)).

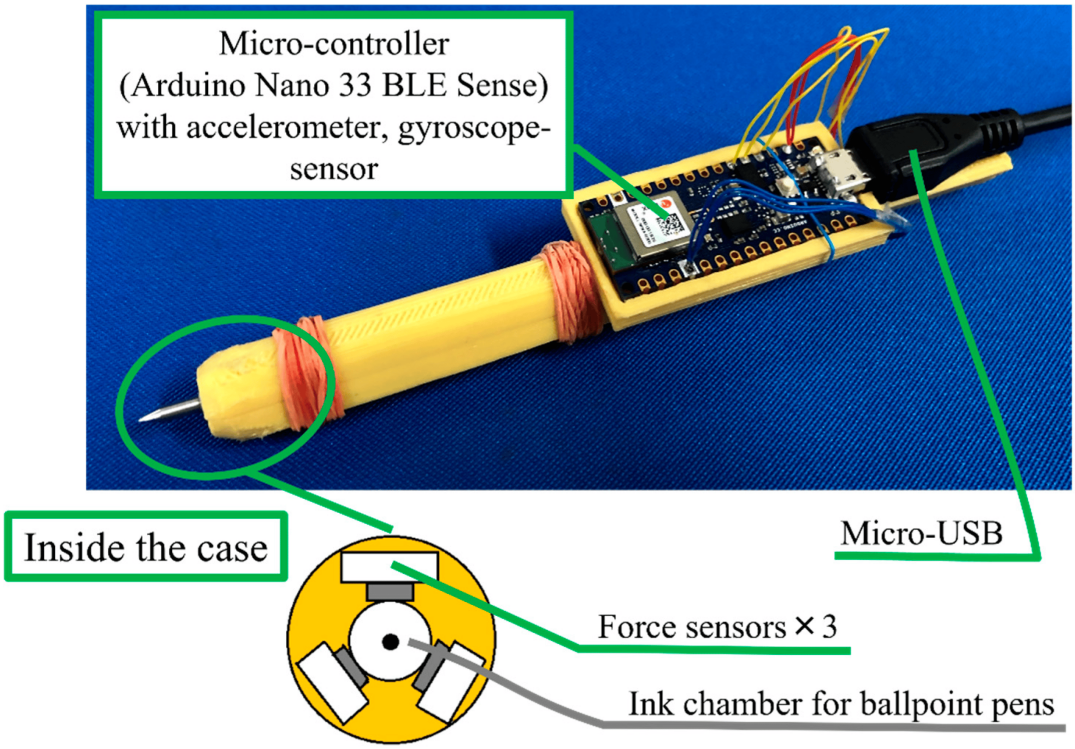


Figure 2.2: The developed smart pen hardware system from Alemayoh et al. (2022)

Figure 2.2 shows the pen, which is equipped with an Arduino Nano microcontroller with an accelerometer and gyroscope sensor, as well as three force sensors in the tip for more accurate handwriting recognition.

Strengths and Weaknesses

The strength of this prototype lies in the sensor fusion, which significantly improves the accuracy. In addition, the force sensor improves data segmentation by distinguishing between writing and idle phases (cf. Alemayoh et al. (2022)). The weakness of the prototype lies in the reduced recognition capability, because the pen is not able to recognize every symbol.

3 IMU-Based Handwriting Tracking

An Inertial Measurement Unit (IMU) is a sensor module that measures and reports an object's specific force, angular rate, and, in some cases, magnetic field. IMUs typically consist of accelerometers, gyroscopes, and sometimes magnetometers. These components help determine an object's orientation and velocity. IMUs are widely used in applications such as motion tracking, navigation, and gesture recognition.

IMUs offer a potential solution to capture pen movements in real time, enabling digital transcription of handwritten text without the need for specialized surfaces or additional tracking devices. This section details the research and experiments conducted using IMU sensors for handwriting tracking, including methodologies, results, limitations, and the ultimate decision to shift toward an optical sensor-based approach.

3.1 Sensor Selection and Setup

We initially selected an IMU sensor (MPU9250) equipped with a 3-axis accelerometer, a 3-axis gyroscope, and a 3-axis magnetometer. The magnetometer can improve orientation tracking by providing an absolute reference for heading. The data from the IMU were collected via an ESP32 microcontroller, which transmitted sensor readings to a computer for processing.

3.1.1 IMU-to-ESP32 Connection

A proper hardware connection is essential for accurate IMU data acquisition. The diagram in Figure 3.1 illustrates how the IMU sensor was connected to the ESP32 microcontroller.

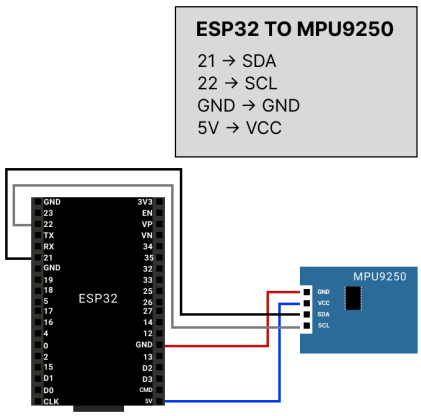


Figure 3.1: IMU Sensor connected to ESP32

3.1.2 Data Acquisition Code

To acquire motion data, the ESP32 continuously reads the IMU accelerometer and gyroscope values, transmitting them via serial communication. The following code configures MPU9250 and ensures reliable data collection:

```
#include <Wire.h>
#include <MPU9250.h>

MPU9250 mpu;

void setup() {
    Serial.begin(115200);
    Wire.begin(21, 22);
    delay(2000);
    mpu.setup(0x68)
    delay(5000);
    mpu.calibrateAccelGyro();
    delay(5000);
    mpu.calibrateMag();
}

void loop() {
    if (mpu.update()) {
        Serial.print(mpu.getGyroX(), 2); Serial.print(",");
        Serial.print(mpu.getGyroY(), 2); Serial.print(",");
        Serial.print(mpu.getGyroZ(), 2); Serial.print(",");
        Serial.print(mpu.getAccX(), 2); Serial.print(",");
        Serial.print(mpu.getAccY(), 2); Serial.print(",");
        Serial.println(mpu.getAccZ(), 2);
    }
}
```

The provided code initializes and configures the MPU9250 IMU sensor, allowing it to capture accelerometer and gyroscope data. It starts by including the necessary libraries (john2zy (2020)) and defining an MPU9250 object. In the setup() function, the serial communication is initialized at 115200 baud, and the I2C communication pins (21 and 22) are set up. The sensor is then configured using mpu.setup(0x68), where 0x68 is the I2C address of the MPU9250. After a short delay, the accelerometer, gyroscope, and magnetometer are calibrated using mpu.calibrateAccelGyro() and mpu.calibrateMag(), ensuring more accurate readings.

In the loop() function, the program continuously checks if new sensor data is available using mpu.update(). If data is updated, it retrieves and prints the gyroscope and accelerometer values in a comma-separated format. The gyroscope readings (in degrees per second) indicate the rotational velocity around the X, Y, and Z axes, while the accelerometer readings (in g) provide the acceleration along these axes. The collected data are sent via serial communication, making it easy to log or analyze in external software like MATLAB or Python.

This approach allows for real-time tracking of motion, which is fundamental in reconstructing handwriting movements. However, IMU sensors are prone to drift and

noise, necessitating additional processing techniques such as filtering and sensor fusion to improve accuracy.

3.1.3 Example Raw Data Output

When running the above code, the IMU sensor outputs raw accelerometer and gyroscope readings in the following format:

```
512, -256, 4096, 125, -654, 8876
```

We chose this format for sending data because it is human-readable and easy to process using standard tools such as MATLAB, Python, or even basic serial monitors. Each value is separated by a comma, which allows for simple parsing in data processing scripts. Although converting these values to a more compact binary format could reduce transmission size and increase efficiency, the impact on USB communication is minimal given the relatively small amount of data being transferred.

USB provides sufficient bandwidth (elektronik kompendium), making it unnecessary to optimize for size at the cost of readability and ease of debugging. Keeping the data in plain text format ensures flexibility and quick testing, especially during the development and debugging phases.

3.2 Data Processing and Tracking Algorithm

The process of tracking handwriting using an IMU sensor begins with data acquisition from the IMU, which consists of an accelerometer and gyroscope. The accelerometer measures linear acceleration in three axes, while the gyroscope captures angular velocity. This raw data is collected at high frequencies, typically around 100Hz, to ensure fine-grained motion capture.

Once the data is recorded, the next step involves filtering to remove high-frequency noise and unwanted variations. A Butterworth (Shouran (2020)) or Kalman filter (Pei (2017)) is applied to smooth the sensor readings and ensure more accurate results. This filtering process is crucial because IMU data is inherently noisy and prone to small fluctuations that can significantly impact trajectory estimation.

Following noise reduction, the orientation of the pen is estimated by fusing accelerometer and gyroscope readings. A complementary filter (Narkhede (2021)) or Madgwick's algorithm (Madgwick (2010)) is commonly used for this purpose, allowing for a more stable estimation of the pen's tilt and rotation. If available, a magnetometer can also be included to correct orientation drift; however, it is susceptible to magnetic interference from surrounding electronic devices.

With the filtered acceleration data available, the velocity of the pen is computed through numerical integration. The integration process accumulates acceleration values over time to estimate velocity in real-time. However, small errors during integration

can accumulate, leading to drift that affects accuracy.

The velocity estimates are then integrated once more to determine the pen's position. This double integration approach reconstructs the trajectory of the handwriting based on movement patterns. However, the lack of an absolute reference makes position estimation particularly challenging. Even minor errors in acceleration measurement can lead to significant deviations in the reconstructed handwriting path.

Finally, the estimated trajectory is visualized as a 2D representation of the handwriting. This output provides insight into how well the IMU-based tracking system performs in reconstructing real pen movements. Despite the best filtering and drift compensation techniques, the reconstructed handwriting is often distorted due to integration errors, sensor drift, and natural hand variations.

The main challenge in IMU-based handwriting tracking is reconstructing the pen's trajectory from raw IMU data. Our approach involved seven steps, which can be seen in figure 3.2.

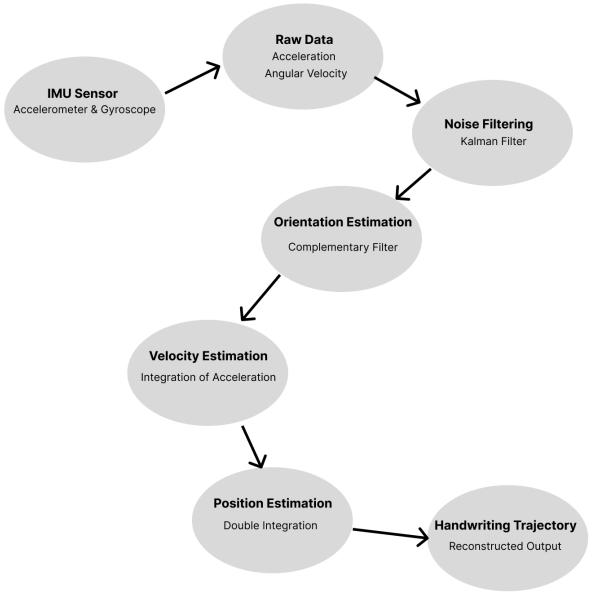


Figure 3.2: Seven-step approach for handwriting tracking

3.2.1 IMU Testing with Pygame Visualization

The IMU Testing with Pygame Visualization section plays an important role in validating the sensor's calibration. By implementing a 3D visualization tool using Pygame and OpenGL, we were able to assess the accuracy of the IMU's orientation tracking before applying it to handwriting reconstruction.

During testing, we observed that the IMU sensor's rotational movements were accurately reflected in the game environment, confirming that our calibration was correct. The real-time visualization demonstrated that the pitch, roll, and yaw values captured by the IMU aligned with actual sensor movements, eliminating potential errors related

to incorrect orientation estimation.

However, while this validation step confirmed the accuracy of rotation tracking, it did not address position tracking. Since handwriting tracking relies on precise position estimation, further processing was required to reconstruct pen movement, which introduced additional challenges such as drift and noise.

The Pygame (Figure 3.2) script reads incoming IMU data, processes pitch, roll, and yaw values, and updates a 3D cube's orientation accordingly. This helps to visually assess how well the IMU is functioning and detect any anomalies.

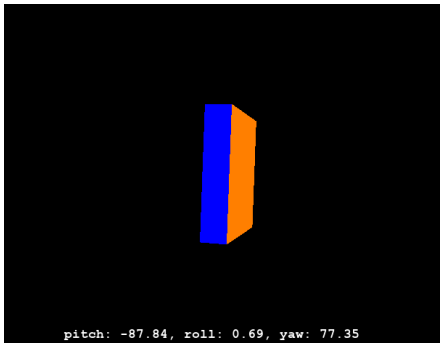


Figure 3.3: Pygame-based IMU visualization

3.2.2 Implementation of Data Processing

In order to reconstruct handwriting movements using IMU data, we needed a robust approach for processing and filtering sensor readings. Since raw IMU data is inherently noisy and subject to drift, it was crucial to apply appropriate filtering and sensor fusion techniques to obtain meaningful position estimates. Our implementation involved collecting real-time IMU data, applying mathematical transformations, and estimating movement trajectories using an Extended Kalman Filter (Ribeiro (2004)).

Extended Kalman Filter (EKF) for Sensor Fusion

The Extended Kalman Filter (EKF) is a widely used algorithm for state estimation in nonlinear systems, particularly in motion tracking and inertial navigation. It extends the classical Kalman Filter (KF) by handling nonlinear relationships between sensor measurements and system states. In our case, We implemented an EKF to integrate IMU data, aiming to mitigate sensor drift. The main function of the EKF is to continuously predict and correct the estimated position and orientation based on sensor readings. This is done in two stages:

- **Prediction Step:** Uses IMU data to estimate the next position and velocity.
- **Update Step:** Refines the estimate by incorporating new sensor readings to correct accumulated errors.

By applying the EKF, we aimed to improve accuracy by reducing drift and compensating for noisy sensor data. The approach is commonly used in robotics, navigation, and motion capture applications.

This section describes our approach, including how data was received, processed, and visualized. We first outline the methodology for obtaining IMU data from the ESP32, followed by the steps taken to process and analyze the data. The final step involves real-time visualization, allowing us to observe how the system interpreted motion in a 3D space.

Despite our efforts to refine the approach, several challenges arose, particularly in mitigating drift and ensuring accurate position tracking. These limitations ultimately influenced our decision to explore alternative tracking methods.

Python

To further analyze the IMU data and try to track handwriting in real-time, we implemented a Python-based solution. This involved receiving the IMU sensor data over a serial connection, processing it using an EKF, and visualizing the computed positions.

Receiving IMU Data via Serial Connection

To retrieve sensor data from the ESP32, we developed a Python script that establishes a serial connection, reads incoming JSON-formatted sensor data, and parses it for further processing. The following implementation ensures stable communication between the ESP32 and a host machine:

```
class SerialWorker:
    def init(self, port, baudrate=115200, timeout=2):
    self.port = port
    self.baudrate = baudrate
    self.timeout = timeout
    self.ser = None

    def connect_to_port(self):
        self.ser = serial.Serial(
            port=self.port,
            baudrate=self.baudrate,
            timeout=self.timeout
        )

    def get_sensor_data(self):
        raw_data = self.ser.readline().decode('utf-8').strip()
        return json.loads(raw_data)  # Parse JSON data
```

The SerialWorker class initializes the serial communication with the specified port and baud rate. The `connect_to_port()` method creates a connection using the serial library, `while get_sensor_data()` reads incoming data, decodes it, and parses it as JSON if it is correctly formatted. This allows for structured data retrieval from the IMU for further processing.

Processing IMU Data and Position Estimation

We developed a pipeline to process IMU data, applying sensor fusion to estimate position while addressing common IMU-related challenges such as drift and noise. The IMU position tracking is based on a modified version of the algorithm provided in the GitHub repository (john2zy (2020)), who also worked on a solution for tracking the position via IMU.

```
app = Flask(name)
socketio = SocketIO(app, async_mode='threading')

PORT = "COM4"
BAUDRATE = 115200

processor = IMUProcessor(sampling=100)
serial_worker = SerialWorker(port=PORT, baudrate=BAUDRATE)
stop_event = threading.Event()

@socketio.on('connect')
def connect():
    stop_event.clear()
    threading.Thread(target=serial_start_listening, daemon=True).start()

@socketio.on('disconnect')
def disconnect():
    stop_event.set()

def serial_start_listening():
    serial_worker.connect_to_port()

    while not stop_event.is_set():
        sensor_data = serial_worker.get_sensor_data()
        if sensor_data and all(key in sensor_data for key in ['gyro', '
    accel', 'mag']):
            processor.initialize(collect_initial_data(sensor_data,
    num_samples=100))
            position = processor.process(sensor_data)
            socketio.emit('position_data', {'x': position[0], 'y':
    position[1], 'z': position[2]})
```

The provided Python code implements real-time IMU data processing and visualization using Flask and Socket.IO. It establishes a communication pipeline between the IMU sensor, data processing algorithms, and a web-based visualization.

- **Flask and Socket.IO Setup:** The script initializes a Flask web application (app) with Socket.IO for real-time data transmission.
- **Serial Connection Initialization:** The SerialWorker class handles communication with the IMU sensor over a serial connection.
- **Threading for Continuous Data Reception:** The `serial_start_listening()` function runs in a separate thread, continuously reading IMU data.

- **Data Processing:** The incoming IMU data is validated to ensure it contains gyroscope, accelerometer, and magnetometer values. The IMUProcessor processes the data using a sensor fusion technique.
- **Real-time Position Estimation:** The computed position is emitted via Socket.IO to update the visualization in a web-based frontend.

This approach allows live tracking of IMU movements.

Position Calculation Using an Extended Kalman Filter

To accurately estimate the position of the pen from IMU data, we applied a sensor fusion approach using an Extended Kalman Filter (EKF). The Kalman filter helps to reduce sensor noise and drift while integrating accelerometer and gyroscope data. The main steps include:

- Sensor Initialization: Collecting a set of initial sensor readings to define a reference state.
- Gravity Compensation: Filtering out the gravity component from accelerometer data.
- Velocity Estimation: Integrating accelerometer readings to estimate velocity.
- Position Estimation: Double integrating acceleration values to compute displacement.
- Error Correction: Using EKF to minimize noise and correct drift.

The following Python class IMUProcessor implements the EKF-based sensor fusion approach. It processes gyroscope, accelerometer, and magnetometer data to estimate movement in 3D space.

```
class IMUProcessor:
    def initialize(self, data):
        w = data[:, self._widx[0]:self._widx[1]]
        a = data[:, self._aidx[0]:self._aidx[1]]
        m = data[:, self._midx[0]:self._midx[1]]

        # Gravity vector estimation
        gn = -a.mean(axis=0)
        g0 = np.linalg.norm(gn)

        # Normalize magnetic field
        mn = m.mean(axis=0) / np.linalg.norm(m.mean(axis=0))

        # Compute sensor noise
        gyro_noise = 100 * np.linalg.norm(w.var(axis=0))
        gyro_bias = w.mean(axis=0)
        acc_noise = 100 * np.linalg.norm(a.var(axis=0))

        self.init_list = (gn, g0, mn, gyro_noise, gyro_bias, acc_noise)
```

```
20
21      def process(self, imu_data):
22          data = np.hstack((imu_data['gyro'], imu_data['accel'], imu_data[
        'mag'])).reshape(1, -1)
23          a_nav, _, _, _ = self.attitudeTrack(data, self.init_list)
24
25          velocities = self.zupt(a_nav, threshold=0.2)
26          positions = self.positionTrack(a_nav, velocities)
27
28          return positions[-1]  # Return final computed position
```

- **Gravity Compensation and Noise Reduction**
  - The initialize() function extracts a stable reference for gravity and magnetic field from initial sensor readings.
  - It computes the variance in gyroscope and accelerometer data to estimate noise.
- **Orientation Estimation**
  - The function attitudeTrack() calculates the sensor's orientation using quaternions, which help convert raw IMU data into a stable global coordinate frame. This step is critical for position estimation.
- **Velocity and Position Calculation**
  - The function zupt() (Zero-Velocity Update) detects stationary phases where acceleration should be zero and removes drift.
  - The function positionTrack() integrates acceleration over time to estimate velocity and subsequently position.

Visualizing Real-Time IMU Data

To display the processed IMU data in an intuitive way, we used a web-based visualization built with Plotly.js and Socket.IO (Figure 3.3). The visualization renders a 3D scatter plot where the tracked pen movement is continuously updated in real-time.

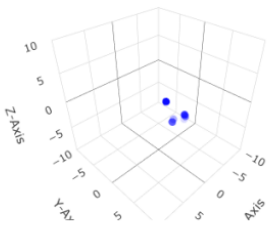


Figure 3.4: web-based visualization

The data is received from the Python backend, processed through the calculations, and then transmitted via a WebSocket connection. The following Python function `serial_start_listening()` continuously reads IMU data from the ESP32, processes it, and sends the computed position to the frontend.

```
def serial_start_listening():
    serial_worker.connect_to_port()

    while not stop_event.is_set():
    if serial_worker.ser and serial_worker.ser.is_open:
        sensor_data = serial_worker.get_sensor_data()
        print(sensor_data)  # Debugging output

        if sensor_data and all(key in sensor_data for key in ['gyro', '
    accel', 'mag']):
            # Collect initial samples for calibration
            initial_data = collect_initial_data(sensor_data, num_samples
    =100)
            processor.initialize(initial_data)

            if sensor_data:
                imu_data = {
                    'gyro': sensor_data['gyro'],
                    'accel': sensor_data['accel'],
                    'mag': sensor_data['mag']
                }

                # Process IMU data to compute position
                position = processor.process(imu_data)

                # Emit position data to the frontend
                socketio.emit('position_data', {
                    'x': position[0],
                    'y': position[1],
                    'z': position[2]
                })
```

- **Establishing Serial Communication**
  - `serial_worker.connect_to_port()` ensures the serial connection to the ESP32 is established.
  - A while loop continuously reads new IMU data unless stopped (`stop_event.is_set()`).
- **Receiving and Filtering IMU Data**
  - `serial_worker.get_sensor_data()` retrieves a single frame of IMU readings.
  - The function checks if all required IMU components (gyro, accel, mag) are available.
  - If valid, it collects initial calibration data (`collect_initial_data(sensor_data, num_samples=100)`) and initializes the sensor processor.
- **Processing Sensor Data**
  - The gyroscope, accelerometer, and magnetometer readings are extracted into `imu_data`.

  - The EKF-based processing pipeline calculates the estimated position of the pen.

- **Transmitting Position Data to the Frontend**
  - The computed position x, y, z is emitted via WebSockets to update the visualization.

The Socket.IO client on the frontend listens for incoming position data and dynamically updates the Plotly.js 3D scatter plot.

```
socket.on('position_data', (msg) => {
    data[0].x.push(msg.x);
    data[0].y.push(msg.y);
    data[0].z.push(msg.z);
    Plotly.update("graph", data, layout);
});
```

The frontend, receives the position data and updates the Plotly.js graph in real time. Each new coordinate is added dynamically, allowing for a continuous visualization of the pen's estimated movement.

This visualization enables a clear representation of how the IMU-based handwriting tracking system interprets movement data. Although the approach demonstrated potential, challenges such as sensor drift, noise accumulation, and limited accuracy prompted us to explore alternative tracking methods.

Matlab

In our exploration of IMU-based position tracking, we found an implementation from the GitHub repository (xioTechnologies (2013)), which successfully demonstrated position tracking using an IMU. The approach used in this implementation processed recorded IMU data stored in a comma-seperated values (CSV) file and then applied sensor fusion techniques to reconstruct position.

To test this method, we recorded our own IMU data and stored it in a CSV file using the following format:

```
Gyroscope X,
Gyroscope Y,
Gyroscope Z,
Accelerometer X,
Accelerometer Y,
Accelerometer Z,
Magnetometer X,
Magnetometer Y,
Magnetometer Z
```

After running this recorded data through the provided MATLAB script, we were able to achieve similar results to those shown in the original implementation (Figure 3.5). Encouraged by these initial results, we aimed to modify the MATLAB script for real-time tracking. This involved capturing live IMU data and processing it dynamically instead of relying on pre-recorded datasets.

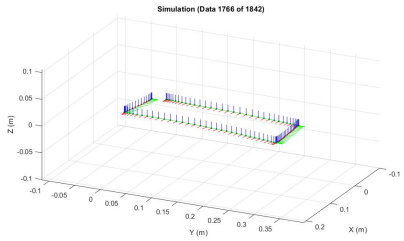


Figure 3.5: Matlab test result for static data

The MATLAB script follows a structured process:

- Importing IMU data – Reads gyroscope, accelerometer, and magnetometer values from a dataset.
- Applying an AHRS algorithm – Computes orientation using the Mahony filter.
- Calculating tilt-compensated acceleration – Converts accelerometer readings to an earth-relative frame.
- Extracting linear acceleration – Removes gravity influence to isolate motion-based acceleration.
- Estimating velocity and position – Integrates acceleration to compute velocity, then integrates velocity to estimate position.
- Filtering to remove drift – High-pass filtering is used to mitigate long-term integration errors.
- Visualizing results – Generates plots for acceleration, velocity, and position over time.

While the approach worked well with pre-recorded data, our attempt to modify it for real-time tracking was unsuccessful (Figure 3.6). MATLAB's approach relies on processing a complete dataset, meaning errors can be corrected post-processing. However, in real-time, errors accumulate without correction, causing severe drift that made tracking unusable.

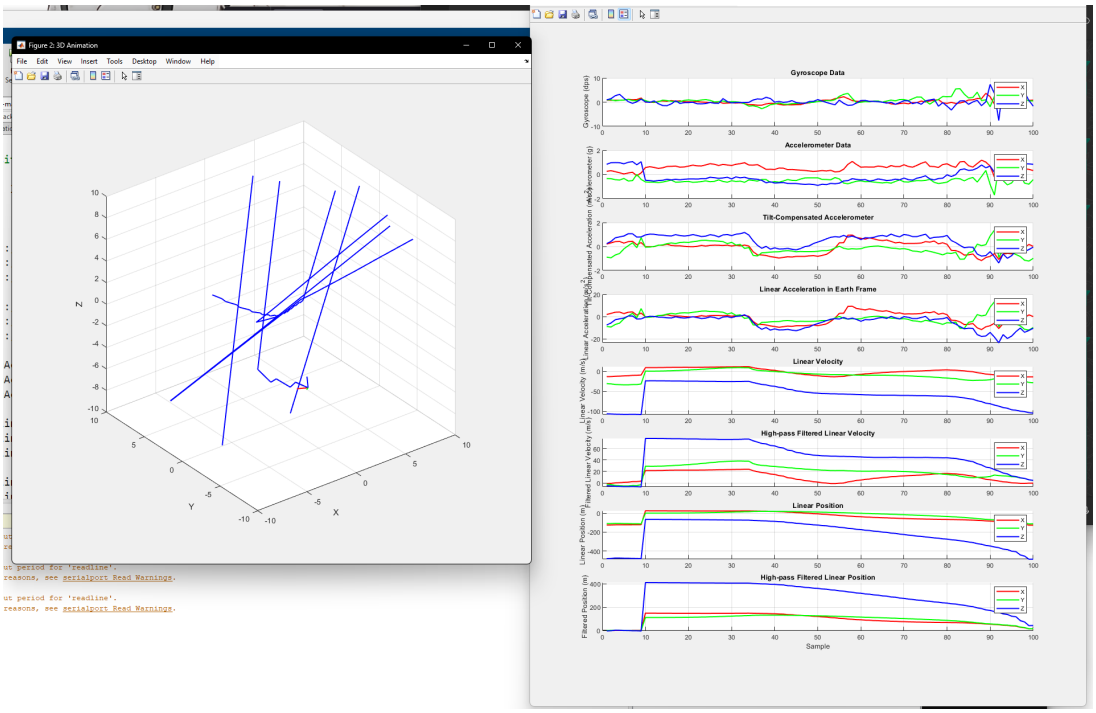


Figure 3.6: Uncontrollable drift in MATLAB-based IMU tracking

The main challenges we faced were:

- Dependence on pre-recorded data – The original implementation relied on a fully captured dataset, making real-time adaptation difficult.
- Drift issues – Despite applying filtering techniques, integration errors accumulated rapidly, leading to significant drift in position estimates.

Due to these issues, we were unable to achieve accurate real-time tracking using this approach, reinforcing our conclusion that IMU-based position tracking is highly susceptible to errors, particularly when attempting live movement reconstruction.

3.2.3 Challenges and Limitations

Altough implementing the EKF-based position estimation and real-time visualization, our approach was ultimately unsuccessful. Even with the use of filtering techniques, cumulative sensor errors resulted in significant trajectory deviations, making accurate position estimation difficult.

Regardless our efforts, the computational demands of sensor fusion, particularly tuning the Kalman filter, limited real-time feasibility. Due to the complexity of the mathematical calculations involved in the EKF and sensor fusion, implementing and optimizing these algorithms posed significant challenges. While we attempted to modify the existing open-source implementation to suit our scenario, debugging and optimizing the filter parameters to reduce drift required expertise beyond our capabilities.

Furthermore, the double integration of accelerometer data, which is essential for position estimation, amplified noise significantly. Even small inaccuracies in sensor readings quickly accumulated into large errors, making the estimated trajectory unreliable. We also observed that magnetometer readings were affected by environmental interference, further contributing to inaccuracies in orientation tracking.

Recognizing the inherent drift in IMU tracking, we pivoted to optical sensors, leveraging their ability to capture motion without cumulative errors.

4 Transition to optical sensor tracking

An alternative approach using an optical sensor from a computer mouse was implemented. This method allows for more precise tracking of pen movement by detecting relative displacement across a writing surface. Additionally, a pressure sensor was integrated to enhance handwriting recognition by detecting writing pressure and distinguishing between writing and non-writing phases. Increased pressure results in thicker strokes, while lifting the pen stops the writing process.

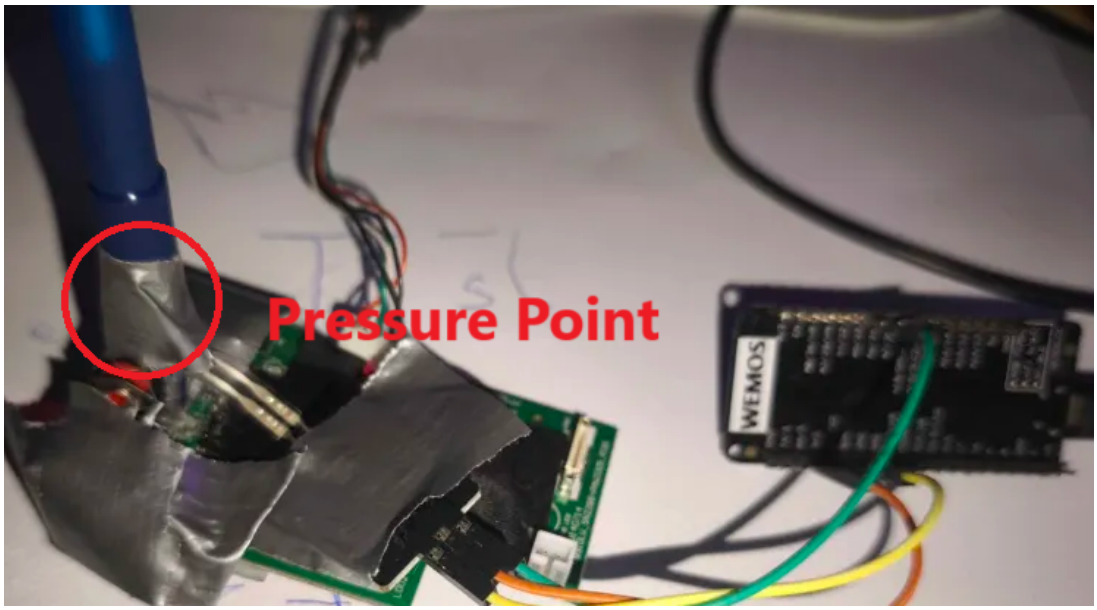


Figure 4.1: Final optical pen prototype powered by a Wemos Lolin32. The pen integrates a Grove pressure sensor to detect writing pressure.

Results Figure 4.2 compares text written on paper with the same text digitized using the optical pen in OneNote. While the digitized handwriting remains legible, deviations in tracking accuracy were observed. The primary limitations stem from the pen's relative tracking nature—an optical sensor determines movement based on changes in position rather than absolute coordinates. Consequently, small shifts in positioning accumulate over time, affecting scale and alignment. Unlike absolute tracking devices (e.g., graphics tablets), which map input directly to a fixed coordinate system, the optical pen cannot maintain a consistent reference point. This limitation prevents precise repositioning after lifting the pen from the writing surface, as movement tracking resumes from an arbitrary starting point.

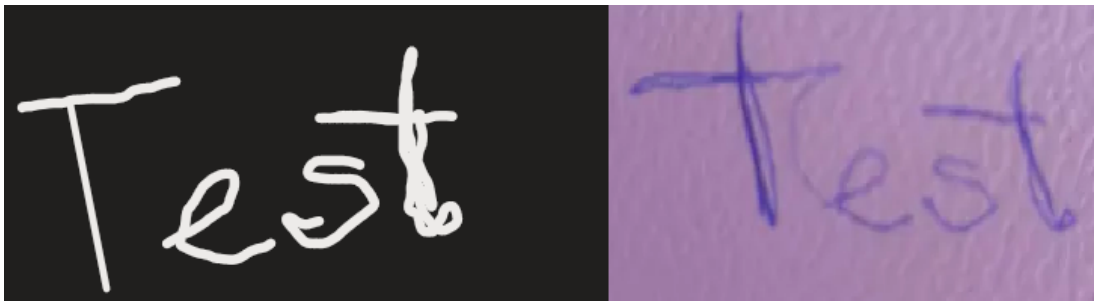


Figure 4.2: Comparison between handwriting and digitized writing.

Despite these drawbacks, optical tracking offers a significant advantage: it allows for direct and responsive cursor movement without the need for complex sensor fusion algorithms or machine learning models. This simplicity makes it an appealing alternative to IMU-based handwriting tracking, as it provides real-time feedback with minimal processing overhead. However, further improvements, such as combining relative tracking with additional reference points or exploring hybrid tracking solutions, could enhance accuracy and usability.

5 Conclusion and Future Work

This study investigated the feasibility of IMU-based handwriting tracking, assessing its accuracy, limitations, and potential as an alternative to traditional optical tracking methods. By integrating an MPU9250 IMU with an ESP32 microcontroller, we developed a data processing pipeline that employed sensor fusion techniques, including an Extended Kalman Filter (EKF), to reconstruct pen movements. Our approach involved real-time processing using Python and post-processing analysis in MATLAB, enabling us to evaluate the performance of IMU-based position estimation.

Despite our efforts, the results demonstrated significant challenges that rendered IMU-based handwriting tracking impractical for accurate digitization. Our results confirm that IMU drift, exacerbated by double integration, remains a fundamental barrier to real-time handwriting tracking. Additionally, the double integration of accelerometer data amplified noise, further exacerbating inaccuracies in position estimation. Although sensor fusion techniques, such as EKF, helped mitigate some errors, they were insufficient to achieve stable and precise tracking over extended writing sessions. The reliance on a magnetometer for orientation correction also introduced variability due to environmental interference, making the system less robust in different settings.

The complexity of sensor fusion techniques, along with the fundamental drift issues, made an IMU-based approach impractical for our application. Without a strong mathematical background, fine-tuning the Kalman filter and interpreting sensor data correctly proved to be a major challenge. Even though we adapted existing open-source solutions, debugging and optimizing them for real-time handwriting tracking required expertise beyond our capabilities.

Given these limitations, we concluded that an IMU-based approach alone is not suitable for real-time handwriting digitization. While it may work under controlled conditions or in conjunction with additional constraints, the standalone use of an IMU for tracking freehand writing remains unreliable. This realization led us to shift our focus toward optical tracking, which provides significantly higher accuracy and stability.

Future Work

The optical pen prototype performed significantly better than the IMU-based approach, but it remains a preliminary prototype with room for improvement. Currently, the pen is functional but not practical for everyday use due to its size and hardware placement. Future work could focus on refining the pen's design, making it more compact and ergonomic. This includes:

- Reducing the size of the optical sensor module to ensure that it does not interfere with natural handwriting posture
- Improving the placement of components so that the PCB is better integrated into the pen's form factor.
- Enhancing writing detection mechanisms by incorporating additional sensors, such as pressure sensors or capacitive touch sensors, to improve stroke detection accuracy.
- Exploring different lens configurations for the optical sensor to allow for better tracking on various writing surfaces, making the system more versatile.

While our study highlights the limitations of IMU-based handwriting tracking, it also contributes valuable insights into the challenges of sensor-based motion estimation. These findings provide a foundation for further research into alternative tracking solutions that balance portability, accuracy, and computational efficiency.

List of Figures

2.1 Sensor Placement in the Pen from Wehbi et al. (2021) 5
2.2 The developed smart pen hardware system from Alemayoh et al. (2022) 6

3.1 IMU Sensor connected to ESP32 7
3.2 Seven-step approach for handwriting tracking 10
3.3 Pygame-based IMU visualization 11
3.4 web-based visualization 15
3.5 Matlab test result for static data 18
3.6 Uncontrollable drift in MATLAB-based IMU tracking 19

4.1 Final optical pen prototype powered by a Wemos Lolin32. The pen integrates a Grove pressure sensor to detect writing pressure. 21
4.2 Comparison between handwriting and digitized writing. 22

Bibliography

Tsige Tadesse Alemayoh, Masaaki Shintani, Jae Hoon Lee, and Shingo Okamoto. Deep-learning-based character recognition from handwriting motion data captured using imu and force sensors. *Sensors*, 22(20):7840, October 2022. ISSN 1424-8220. doi: 10.3390/s22207840. URL http://dx.doi.org/10.3390/s22207840.

elektronik kompendium. Usb 3.0 / 3.1 / 3.2 / superspeed-usb. URL https://www.elektronik-kompendium.de/sites/com/1310061.htm. Accessed: 2025-02-27.

john2zy. Imu-position-tracking, 2020. URL https://github.com/john2zy/IMU-Position-Tracking/tree/master. Accessed: 2025-02-27.

Sebastian O.H. Madgwick. An efficient orientation filter for inertial and inertial/magnetic sensor arrays, 2010. URL https://courses.cs.washington.edu/courses/cse466/14au/labs/l4/madgwick_internal_report.pdf. Accessed: 2025-02-27.

Parag Narkhede. Cascaded complementary filter architecture for sensor fusion in attitude estimation, 2021. URL https://arxiv.org/pdf/2107.03970. Accessed: 2025-02-27.

Yan Pei. An elementary introduction to kalman filtering, 2017. URL https://arxiv.org/pdf/1710.04055. Accessed: 2025-02-27.

Isabel Ribeiro. Kalman and extended kalman filters: Concept, derivation and properties, 2004. URL https://www.researchgate.net/publication/2888846_Kalman_and_Extended_Kalman_Filters_Concept_Derivation_and_Properties. Accessed: 2025-02-27.

Mokhtar Shouran. Design and implementation of butterworth filter, 2020. URL https://www.researchgate.net/publication/344250131_Design_and_Implementation_of_Butterworth_Filter. Accessed: 2025-02-27.

Mohamad Wehbi, Tim Hamann, Jens Barth, Peter Kaempf, Dario Zanca, and Bjoern Eskofier. Towards an imu-based pen online handwriting recognizer, 2021. URL https://arxiv.org/abs/2105.12434.

xioTechnologies. Oscillatory motion tracking with x-imu, 2013. URL https://github.com/xioTechnologies/Oscillatory-Motion-Tracking-With-x-IMU. Accessed: 2025-02-27.